

集美大学

微机原理与接口技术课程设计

学　　院　海洋信息工程学院　班　　级　通信 2414

姓　　名　谢欣怡　学　　号　202421303109

姓　　名　李哲　学　　号　202421303129

姓　　名　林祖澎　学　　号　202421303116

姓　　名　吴智聪　学　　号　202421303115

小 组 号　111　成　　绩

指导老师　邓旭初老师 方怡冰老师

2026 年 6 月 11 日

微机原理与接口技术课程设计设计报告

小车自动行驶系统

一、任务要求

（1）小车设计目的

本项目旨在设计一套基于 PIC16F877A 单片机 控制的自动行驶小车系统，能够在指定路径上完成精确循迹、定点停车、路径切换等任务，模拟真实自动驾驶行为，提高机器人技术应用水平。

（2）场地与路径要求

场地面积不小于 220cm × 120cm，使用白色哑光喷绘布制作。

包含两个对称半圆弧线，半径 40cm，线宽约 1.8cm，黑色。

弧线四个顶点定义为 A、B、C、D，除半圆弧外不得添加任何标记。

（3）行驶功能要求

1.基础行驶（A→B）

小车从 A 点自动行驶到 B 点停车

停车时有声光提示

用时≤5 秒

2.完整一圈（A→B→C→D→A）

路径：A→B（直线）→C（半弧）→D（直线）→A（半弧）

每经过一个顶点，声光提示一次。

用时≤30 秒

3.交叉一圈（A→C→B→D→A）

路径：A→C（半弧）→B（直线）→D（半弧）→A（直线）

每经过一个顶点，声光提示一次。

用时≤ 40 秒

4.连续四圈（按路径 3 行驶）

按第 3 条路径自动行驶 4 圈 后停车。

用时越少越好

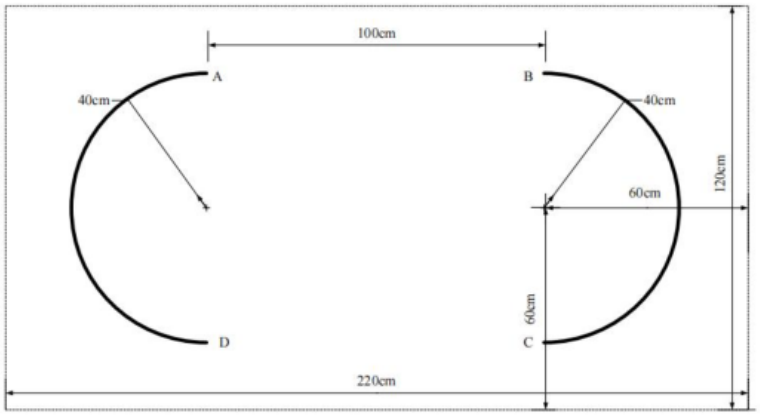


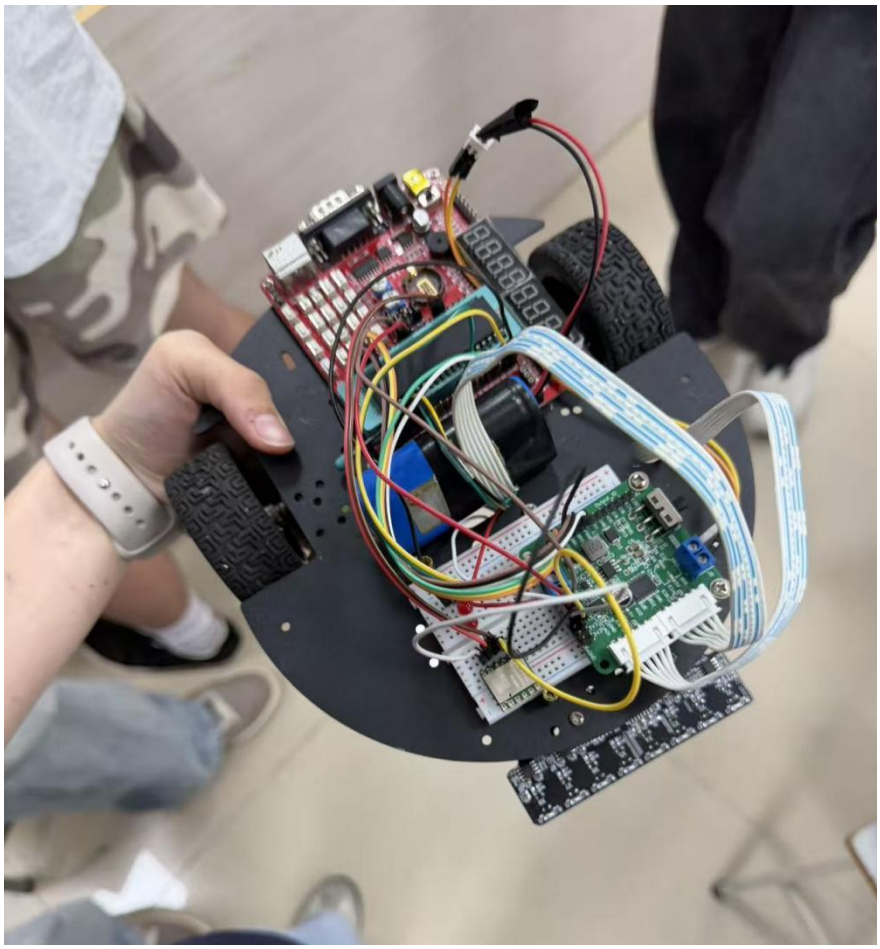
图 1 小车行驶场地示意图

二、设计器材使用

1)锂电池
2)TB6612FNG 电机驱动模块
3)八路循迹模块(八路灰度传感器)
4)二驱小车套件
5) PIC16F877A 单片机
6) JY62 陀螺仪
7)杜邦线、洞洞板等
8）LED 灯

三、原理及思路

3.1 硬件部分



3.1.1 硬件部分介绍

1.主控单元：PIC16F877A 单片机

作为小车的核心控制单元，负责所有信号处理、逻辑决策、中断响应与 PWM 输出。需合理分配各功能模块（循迹信号读取、陀螺仪信号解析、电机 PWM 控制、声光提示等）至不同中断与标志变量，避免全部放在主循环中导致任务冲突。

2.电机驱动模块：TB6612FNG

用于控制小车左右电机的正转、反转及速度调节，接收来自 PIC16F877A 的 PWM 信号（通过 CCP1/CCP2 + TMR2 产生）。具备高电压承受能力和稳定驱动性能，适合精确速度控制需求。

3.姿态与方向传感器：JY62 陀螺仪

用于在 AB、CD、AC、BD 段（即无黑线指引的直线段或斜线段）提供平面角度信号。小车通过解析 JY62 的航向角数据实现直线行驶与方向控制。

注意：陀螺仪必须与车体牢固连接，确保角度数据准确。

4.循迹模块：八路灰度传感器（八路循迹模块）

用于 BC 和 DA 段（即带有黑线的半圆弧路径）的路面灰度检测。通过实时反馈小车相对于黑线的偏移量，主控动态调整方向，实现稳定循迹。

注意：车体与循迹模块至少有两个固定点，避免行驶中晃动。

3.1.2 模块搭建

1.组件连接方式

按照系统设计图纸，将各硬件模块逐一连接至主控单元（PIC16F877A 单片机）。重点确保电机驱动模块、循迹模块、陀螺仪及电源配置连接正确。

2.硬件搭建步骤

第一步：将 **TB6612FNG 电机驱动模块** 连接至主控单元，确保电源（锂电池）接入无误，并检查电机输出端与左右电机的对应关系。

第二步：将 **八路循迹模块** 正确安装于小车底部，要求与地面保持适当距离（通常 1–2cm），并至少使用两个固定点与车体牢固连接，防止行驶中晃动。

第三步：将 **JY62 陀螺仪** 固定于车体中心位置，确保与车体刚性连接，避免相对位移影响角度测量精度。

第四步：完成所有模块连接后，检查电源开关及各模块间线路，使用万用表电阻档测量关键连接点，确认无短路、断路情况。

第五步：上电测试，逐一验证各模块工作状态（如传感器数据是否正常、电机是否可控），确保系统整体运行稳定。

引脚定义

JY62 陀螺仪接线表

JY62 引脚	PIC 单片机引脚
TX	RC7 (USART RX)
GND	GND
VCC	3.3V

TB6612 电机驱动接线表

TB6612 引脚	PIC 单片机引脚
PWMA	RC2 (CCP1)
AIN1	RB4
AIN2	RB5
PWMB	RC1 (CCP2)
BIN1	RB6
BIN2	RB7
STBY	VCC

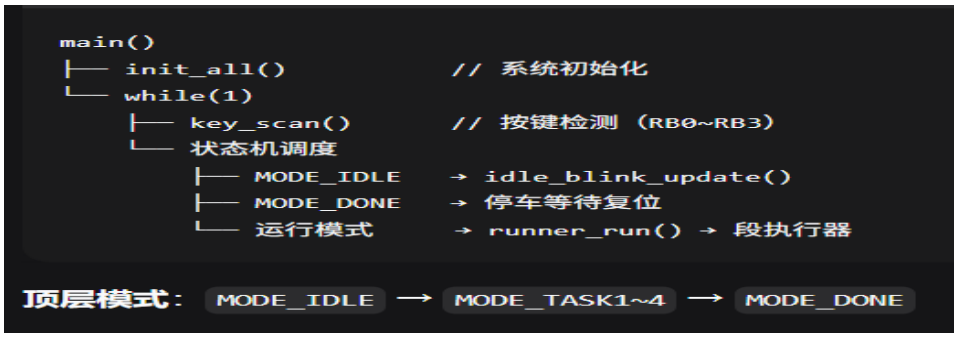
八路灰度循迹模块接线表

灰度模块引脚	PIC 单片机引脚
OUT	RA0
AD0	RA1
AD1	RA2
AD2	RA3

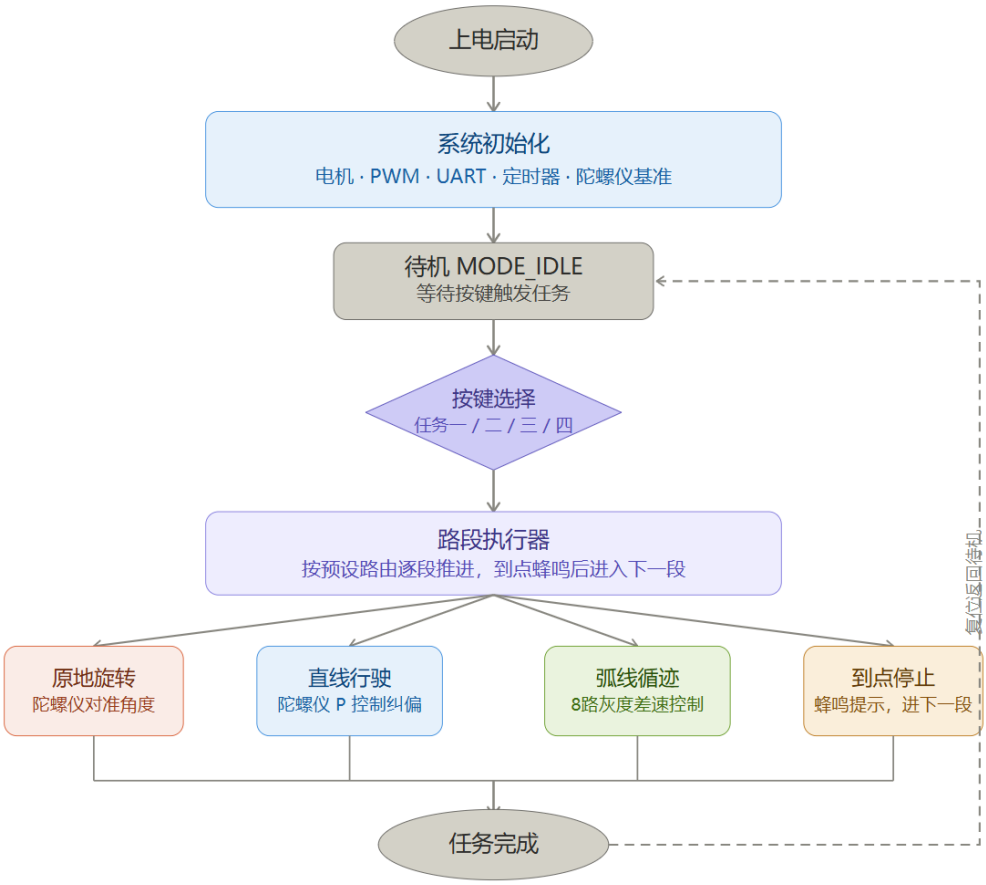
声光提示模块接线表

模块引脚	PIC 单片机引脚
RE0	蜂鸣器
RD0~RD2	LED

3.2 软件部分



系统流程框图：



3.2.1 初始参数定义、引脚定义、路径设置

软件开始部分主要完成三类基础配置：一是可调控制参数，二是单片机与外设的引脚映射，三是任务路径的数据化描述。这一部分相当于整个程序的“配置层”，把后续控制算法需要反复调用或调试的内容统一放在程序前部，便于在硬件接线、场地尺寸或小车速度发生变化时快速修改。

（1）可调参数区：程序用宏定义集中设置直线基础速度 BASE_PWM、弧线内外轮速度 ARC_FAST/ARC_SLOW、原地旋转速度 SPIN_SPEED、直线 P 控制系数 KP、直线修正死区、最大修正量、弧线入弧预转角、连续丢线确认次数等参数。这样做的优点是把“调车参数”和“控制逻辑”分开，现场调试时只需要调整数值，不需要改动函数内部判断流程，可以减少调试时引入新错误的可能。

（2）引脚定义区：程序通过 AIN1、AIN2、BIN1、BIN2 定义 TB6612 电机驱动方向引脚，通过 GRAY_OUT、GRAY_AD0、GRAY_AD1、GRAY_AD2 定义八路灰度传感器地址选择与输出引脚，通过 BTN_RESET、BTN_TASK1、BTN_TASK2、BTN_TASK3 定义按键输入，通过 BUZZER 和 LED_TASK 定义声光提示接口。引脚统一宏定义后，主程序只面向功能名称编程，若实际接线调整，只需要修改对应宏定义，后面的电机、按键、传感器函数不需要整体重写。

（3）数据结构区：程序定义了 CarMode 表示小车当前处于待机、任务一、任务二、任务三、任务四或完成状态；定义 SegType 表示路径段类型，包括原地旋转 SEG_SPIN、直线 SEG_STRAIGHT、弧线 SEG_ARC 和停止 SEG_STOP；

再通过 Segment 结构体把路径段类型、旋转目标角和到达点标识组合起来。这样可以把复杂路线拆成一个个标准动作段，使路径执行器只需要根据当前段类型调用对应函数。

（4）路径表区：任务一、任务二、任务三和任务四的运动路线都用 Segment 数组描述。例如任务一为直线到 B 点后停止；任务二为 A 到 B、弧线到 C、直线到 D、弧线回 A；任务三先原地旋转对准 AC 方向，再直线到 C、弧线到 B、直线到 D、弧线回 A；任务四复用任务三路径并循环四圈。任务路线以表格数据形式存在，控制算法与路线描述分离，后期若增加新任务，只需增加新的路径数组并在启动函数中选择即可。

通过上述设计，程序前半部分完成了硬件抽象、控制参数管理和任务路径抽象，使后续模块可以围绕“当前模式、当前路径段、当前传感器状态”进行控制，整体结构更加清晰，也更适合课程设计中的反复调试。

```
#include <pic.h>

__CONFIG(0xFF29);

/* ============================================================
 * 可调参数（根据实测修改）
 * ============================================================ */
#define BASE_PWM         500     /* 直线基础速度 */
#define ARC_FAST         530     /* 弧线外侧轮速度 */
#define ARC_SLOW         220     /* 弧线内侧轮速度 */
#define ARC_HARD_FAST    620     /* 边缘检测到线时外侧轮强修正速度 */
#define ARC_HARD_SLOW    120     /* 边缘检测到线时内侧轮强修正速度 */
#define SPIN_SPEED       240     /* 原地旋转速度 */
#define KP               3       /* 直线P控制系数 */
#define STRAIGHT_DEADBAND 80     /* 约0.4度内不修正，避免抖动 */
#define STRAIGHT_ERR_DIV  64     /* Yaw raw缩放，避免小偏差过度修正 */
#define STRAIGHT_MAX_CORR 180    /* 单侧最大PWM修正量 */
#define ARC_MIN_YAW_RAW  26400 /* 弧线至少转过约145度后才允许判端点 */
#define ARC_LOST_CONFIRM 8     /* 全白连续确认次数，数值越小到点/停止越快 */
#define ARC_ENTRY_YAW_RAW 1200 /* 任务二刚入弧先右转约7度，避免B点后直冲 */
#define ARC_ENTRY_CB_YAW_RAW 1800 /* 任务三C->B刚入弧先左转约10度 */
#define ALIGN_BASE_PWM   320   /* C->D开头低速修正速度 */
#define ALIGN_MAX_CORR   220   /* C->D开头最大PWM修正量 */
#define ALIGN_DONE_RAW   900   /* 约5度内退出强修正 */
#define STRAIGHT_IGNORE_MS 700 /* 进入直线后先忽略黑线，防止重复判点 */
#define GRAY_SETTLE_LOOPS 80  /* 地址切换后等待OUT稳定，防止读到假全白 */
#define GRAY_SAMPLE_GAP   20  /* 同一路多次采样间隔 */
#define GRAY_SAMPLE_COUNT 3   /* 每一路采样次数 */
#define GRAY_SAMPLE_HIGH  2   /* 多数为高才认为该路检测到黑线 */
#define GRAY_BLACK_HIGH   1   /* 1=黑线输出高电平，0=黑线输出低电平 */


/* AC方向偏角，JY62原始值，顺时针为负
 * 若实测方向相反，改为正值 +7396 */
#define AC_SPIN_TARGET   (-6400)

/* 旋转到位容差：约±3° = 3*32768/180 ≈ ±546 raw */
#define SPIN_TOLERANCE   750
```

```c
/* 声光提示持续时间(ms) */
#define BEEP_TIME_MS    200

/* 长按任务三按键触发任务四的时间(ms) */
#define LONG_PRESS_MS   1000

/* ============================================================
 * 引脚定义
 * ============================================================ */
/* 电机方向 */
#define AIN1    RB4
#define AIN2    RB5
#define BIN1    RB6
#define BIN2    RB7

/* 灰度传感器（RA口，OUT/AD0/AD1/AD2地址选择式） */
#define GRAY_OUT    RA0
#define GRAY_AD0    RA1
#define GRAY_AD1    RA2
#define GRAY_AD2    RA3

/* 按键（RB口，低电平有效，内部上拉） */
#define BTN_RESET   RB0     /* 停止/复位 */
#define BTN_TASK1   RB1     /* 启动任务一 */
#define BTN_TASK2   RB2     /* 启动任务二 */
#define BTN_TASK3   RB3     /* 启动任务三（长按=任务四） */

/* 声光 */
#define BUZZER      RE0
#define LED_TASK    PORTD       /* RD0~RD3任务指示，RD4~RD7到点提示 */
```

```c
/* ============================================================
 * 顶层模式定义
 * ============================================================ */
typedef enum {
    MODE_IDLE = 0,
    MODE_TASK1,
    MODE_TASK2,
    MODE_TASK3,
    MODE_TASK4,
    MODE_DONE
} CarMode;

/* ============================================================
 * 路径段类型
 * ============================================================ */
typedef enum {
    SEG_SPIN = 0,   /* 原地旋转到目标角度 */
    SEG_STRAIGHT,   /* 直线（陀螺仪P控制） */
    SEG_ARC,        /* 弧线（循迹跟线） */
    SEG_STOP        /* 终止 */
} SegType;

typedef struct {
    SegType type;
    int     spin_target;    /* SEG_SPIN: 目标偏转角(raw)，相对于初始基准 */
    char    label;          /* 到达点标识，声光提示用 */
} Segment;

/* ============================================================
 * 路径表
 * ============================================================ */
```

```
/* 任务一：A→B */
const Segment route1[] = {
    {SEG_STRAIGHT, 0,                'B'},
    {SEG_STOP,     0,                'B'}
};
#define ROUTE1_LEN 2

/* 任务二：A→B→弧BC→C→D→弧DA→A */
const Segment route2[] = {
    {SEG_STRAIGHT, 0,                'B'},
    {SEG_ARC,      0,                'C'},
    {SEG_STRAIGHT, 0,                'D'},
    {SEG_ARC,      0,                'A'},
    {SEG_STOP,     0,                'A'}
};
#define ROUTE2_LEN 5

/* 任务三/四：A→旋转→直线C→弧CB→B→直线D→弧DA→A */
const Segment route3[] = {
    {SEG_SPIN,     AC_SPIN_TARGET,   'A'},    /* 原地旋转对准AC方向 */
    {SEG_STRAIGHT, 0,                'C'},    /* 直线到C */
    {SEG_ARC,      0,                'B'},    /* 弧线CB */
    {SEG_STRAIGHT, 0,                'D'},    /* 直线到D */
    {SEG_ARC,      0,                'A'},    /* 弧线DA */
    {SEG_STOP,     0,                'A'}
};
#define ROUTE3_LEN 6
```

3.2.2 硬件初始化模块

硬件初始化模块由 pwm_init()、motor_init()、uart_init()、timer1_init()和 init_all()组成，在小车上电后首先执行。该模块的作用是把 PIC16F877A 的端口方向、PWM 输出、串口接收、定时中断和初始输出状态全部配置完成，保证主循环开始运行时，各外设已经处于确定状态。

pwm_init()负责配置 Timer2 和 CCP1、CCP2 模块，产生用于左右电机调速的 PWM 信号。程序中 PWM 频率设置为约 4kHz，占空比采用 0 到 1000 的控制范围，后续 motor_forward()、motor_spin_left()等函数只需要写入速度值，就可以转换成对应的 PWM 寄存器值。该设计使电机调速与底层寄存器配置分离。

motor_init()负责把 RB4 到 RB7 配置为电机方向控制输出，并在初始化结束时调用 motor_stop()让小车保持停止状态。这样可以避免系统刚上电或复位瞬间电机误动作，提高调试和运行安全性。

uart_init()负责配置 RC7 为串口接收端、RC6 为发送端，设置 9600bps 异步通信，并开启 USART 接收中断。JY62 陀螺仪不断向单片机发送姿态数据，程序在中断服务函数中按 0x55、0x53 帧头解析 Yaw 偏航角，并设置新数据标志 g_new_frame，供直线控制和旋转控制使用。

timer1_init()负责配置定时器 1 产生约 1ms 的周期中断，形成系统统一时基 g_timer_ms。按键去抖、长按识别、待机 LED 闪烁、到点蜂鸣持续时间、直线段刚进入时忽略黑线的保护时间，都依靠这一毫秒计数完成。使用统一时基后，程序不需要大量阻塞延时，主循环能够持续扫描按键并执行路径任务。

init_all()是总初始化函数，负责关闭比较器、将 RA 口设置为数字 I/O、配置各端口输入输出方向、打开 RB 口内部上拉、关闭蜂鸣器、熄灭 LED，并依次调用 PWM、电机、串口和定时器初始化函数。初始化完成后，小车进入待机模式，RD0 按固定周期闪烁，提示系统已经准备好接收任务按键。

```
/* 顶层模式 */
volatile CarMode g_mode = MODE_IDLE;

/* 路径执行器 */
const Segment   *g_route     = route1;
unsigned char    g_route_len = ROUTE1_LEN;
unsigned char    g_step      = 0;
unsigned char    g_lap       = 0;       /* 任务四圈数计数 */
unsigned char    g_arc_lost_cnt = 0;    /* 弧线端点连续确认计数 */
char             g_arc_turn_ready = 0;  /* 弧线转角已达到端点判定阈值 */
signed char      g_gray_last_side = 1;   /* 灰度丢线时沿上次方向找线 */
char             g_straight_wait_clear = 0; /* 直线段先等离开上一条线 */
unsigned int     g_straight_start_ms = 0;

/* 段内子状态（用于BEEP后跳转） */
/* SEG_BEEP结束后直接step++，不需要额外变量 */

/* 陀螺仪（USART中断更新） */
volatile int  g_yaw       = 0;
volatile char g_new_frame = 0;

/* USART接收缓冲 */
#define FRAME_LEN 11
volatile unsigned char g_uart_buf[FRAME_LEN];
volatile unsigned char g_uart_idx  = 0;
volatile char          g_uart_sync = 0;

/* 定时器（TMR1中断更新） */
volatile unsigned int g_timer_ms = 0;

/* 段执行基准角 */
int g_yaw_ref      = 0;     /* 直线段/旋转段：出发时的Yaw基准 */
int g_arc_yaw_ref  = 0;     /* 弧线段：入弧时的Yaw基准 */
int g_ab_yaw_ref   = 0;     /* 任务二AB实际直线基准 */
int g_bd_yaw_ref   = 0;     /* 任务三B->D理论直线基准 */
char g_align_cd    = 0;     /* 1=C->D开头低速强修正 */
```

```c
/* BEEP计时 */
unsigned int g_beep_start = 0;
char         g_in_beep    = 0;  /* 1=正在BEEP中 */


/* 待机闪烁计时 */
unsigned int g_idle_blink_ts = 0;
char         g_idle_blink_state = 0;    /* 0=灭，1=亮 */

/* 旋转基准 */
int g_spin_yaw_ref = 0;

/* 初始绝对Yaw基准（上电时记录，用于计算AC段绝对目标角） */
int g_yaw_init = 0;

/* ============================================================
 * 工具函数
 * ============================================================ */
int abs_val(int x) { return (x < 0) ? -x : x; }

/* Yaw角差，归一化到 -32768~+32767 (对应-180°~+180°) */
int yaw_diff(int current, int ref)
{
    long diff = (long)current - (long)ref;
    if (diff >  32767L) diff -= 65536L;
    if (diff < -32768L) diff += 65536L;
    return (int)diff;
}

int yaw_add_raw(int ref, int delta)
{
    int value = ref + delta;
    if (delta > 0 && value < ref) value -= 32768;
    if (delta < 0 && value > ref) value += 32768;
    return value;
}

/* 返回与ref相差180度的Yaw原始值 */
```

```c
int yaw_opposite(int ref)
{
    if (ref >= 0) return ref - 32767 - 1;
    return ref + 32767 + 1;
}

unsigned int yaw_abs_diff(int current, int ref)
{
    long diff = (long)current - (long)ref;
    if (diff >  32767L) diff -= 65536L;
    if (diff < -32768L) diff += 65536L;
    if (diff < 0) diff = -diff;
    return (unsigned int)diff;
}

/* ============================================================
 * PWM 初始化
 * ============================================================ */
void pwm_init(void)
{
    TRISC1  = 0;
    TRISC2  = 0;
    PR2     = 249;
    T2CON   = 0x04;
    CCP1CON = 0x0C;
    CCPR1L  = 0;
    CCP2CON = 0x0C;
    CCPR2L  = 0;
}

void pwm_set(unsigned int left, unsigned int right)
{
    if (left  > 1000) left  = 1000;
    if (right > 1000) right = 1000;
    CCPR1L  = (unsigned char)(left  >> 2);
    CCP1CON = (CCP1CON & 0xCF) | (unsigned char)((left  & 0x03) << 4);
    CCPR2L  = (unsigned char)(right >> 2);
    CCP2CON = (CCP2CON & 0xCF) | (unsigned char)((right & 0x03) << 4);
}
```

```c
/* ============================================================
 * 电机控制
 * ============================================================ */
/* 两轮前进 */
void motor_forward(unsigned int l, unsigned int r)
{
    AIN1 = 1; AIN2 = 0;
    BIN1 = 1; BIN2 = 0;
    pwm_set(l, r);
}

/* 原地左转：左轮后退，右轮前进 */
void motor_spin_left(unsigned int spd)
{
    AIN1 = 0; AIN2 = 1;    /* 左轮反转 */
    BIN1 = 1; BIN2 = 0;    /* 右轮正转 */
    pwm_set(spd, spd);
}

/* 原地右转：左轮前进，右轮后退 */
void motor_spin_right(unsigned int spd)
{
    AIN1 = 1; AIN2 = 0;    /* 左轮正转 */
    BIN1 = 0; BIN2 = 1;    /* 右轮反转 */
    pwm_set(spd, spd);
}

void motor_stop(void)
{
    AIN1 = 0; AIN2 = 0;
    BIN1 = 0; BIN2 = 0;
    pwm_set(0, 0);
}

void motor_init(void)
{
    TRISB4 = 0; TRISB5 = 0;
    TRISB6 = 0; TRISB7 = 0;
    motor_stop();
}
```

```c
/* ============================================================
 * USART 初始化（9600bps，接收中断）
 * ============================================================ */
void uart_init(void)
{
    TRISC7 = 1;
    TRISC6 = 0;
    SPBRG  = 25;
    BRGH   = 1;
    SYNC   = 0;
    SPEN   = 1;
    CREN   = 1;
    TXEN   = 0;
    RCIE   = 1;
    PEIE   = 1;
    GIE    = 1;
}

/* ============================================================
 * TMR1 初始化（1ms中断）
 * ============================================================ */
void timer1_init(void)
{
    T1CON  = 0x00;
    TMR1H  = 0xFC;
    TMR1L  = 0x18;
    TMR1IE = 1;
    TMR1ON = 1;
}
```

```c
/* ============================================================
 * 中断服务程序
 * ============================================================ */
void interrupt isr(void)
{
    /* USART接收：逐字节拼帧 */
    if (RCIF && RCIE)
    {
        unsigned char rx;
        unsigned char i;
        unsigned char sum;

        if (OERR) { CREN = 0; CREN = 1; }
        rx = RCREG;

        if (!g_uart_sync)
        {
            if (rx == 0x55) {
                g_uart_buf[0] = 0x55;
                g_uart_idx    = 1;
                g_uart_sync   = 1;
            }
        }
        else
        {
            g_uart_buf[g_uart_idx++] = rx;

            if (g_uart_idx == 2) {
                if (rx != 0x53) {
                    g_uart_sync = 0;
                    g_uart_idx  = 0;
                }
            }
            else if (g_uart_idx >= FRAME_LEN)
            {
                sum = 0;
                for (i = 0; i < FRAME_LEN - 1; i++)
                    sum += g_uart_buf[i];

                if (sum == g_uart_buf[FRAME_LEN - 1]) {
```

```
                                    | g_uart_buf[6]);
                g_new_frame = 1;
            }
            g_uart_sync = 0;
            g_uart_idx  = 0;
        }
    }
}

/* TMR1: 1ms计时 */
if (TMR1IF && TMR1IE)
{
    TMR1IF = 0;
    TMR1H  = 0xFC;
    TMR1L  = 0x18;
    g_timer_ms++;
}
```

3.2.3 灰度传感器模块

灰度传感器模块负责读取八路地址选择式循迹传感器的状态，核心函数为gray_read()、gray_line_detected()、gray_any_line_detected()、gray_line_side()和gray_line_side_from_data()。本设计使用 3 根地址线 AD0、AD1、AD2 选择 8 个传感器通道，再通过一个 OUT 引脚读取当前通道的检测结果，因此能够在节省单片机 I/O 口的情况下完成八路检测。

gray_read()的工作过程是：程序依次输出 0 到 7 的三位地址，等待 OUT 信号稳定后，对每一路进行多次采样；若多数采样结果满足黑线电平条件，就把该通道对应的二进制位写入返回值。最终返回的 8 位数据中，每一位代表一路传感器是否检测到黑线。程序设置 GRAY_SETTLE_LOOPS、GRAY_SAMPLE_GAP、GRAY_SAMPLE_COUNT 和 GRAY_SAMPLE_HIGH，是为了减小地址切换、电平抖动和瞬时误读对循迹判断的影响。

gray_line_detected()用于判断当前是否至少有一路传感器检测到黑线，主要用于直线段的到点识别；gray_any_line_detected()用于判断是否存在有效线迹；gray_line_side()则进一步分析黑线位于小车左侧、中间还是右侧，并返回-2、-1、0、1、2 等偏移等级。返回值越偏向两侧，说明小车偏离轨迹越明显，需要更强的差速修正。

在偏移判断中，程序把 8 路传感器划分为左侧、中间和右侧区域。如果中间传感器检测到黑线或左右检测数量接近，则认为小车基本居中，返回 0；若左侧检测数量更多，则说明黑线偏左，小车需要向左修正；若右侧检测数量更多，则向右修正；当最左或最右边缘传感器触发时，程序返回强修正等级，用更大的左右轮速度差把车拉回轨迹。

当短时间内读不到黑线时，程序不会立即认为小车已经到达终点，而是保留 g_gray_last_side 作为上一次偏移方向。这样在弧线段短暂丢线时，小车仍会沿上一次判断方向继续修正，避免因为单次全白数据导致左右反复抖动或提前停车。

```
/* ============================================================
 * 八路灰度传感器（地址选择式）
 * AD2~AD0选择通道0~7，OUT输出该通道状态
 * S1=bit7 ... S8=bit0
 * 检测到黑线=1
 * ============================================================ */
unsigned char gray_read(void)
{
    unsigned char data = 0;
    unsigned char i;
    unsigned char sample;
    unsigned char high_cnt;
    volatile unsigned char settle;

    TRISA0 = 1;     /* RA0/OUT 输入 */
    TRISA1 = 0;     /* RA1/AD0 输出 */
    TRISA2 = 0;     /* RA2/AD1 输出 */
    TRISA3 = 0;     /* RA3/AD2 输出 */

    for (i = 0; i < 8; i++) {
        GRAY_AD0 = (i & 0x01) ? 1 : 0;
        GRAY_AD1 = (i & 0x02) ? 1 : 0;
        GRAY_AD2 = (i & 0x04) ? 1 : 0;

        for (settle = 0; settle < GRAY_SETTLE_LOOPS; settle++) { ; }

        high_cnt = 0;
        for (sample = 0; sample < GRAY_SAMPLE_COUNT; sample++) {
            if (GRAY_OUT)
                high_cnt++;
            for (settle = 0; settle < GRAY_SAMPLE_GAP; settle++) { ; }
        }

        if (GRAY_BLACK_HIGH)
        {
            if (high_cnt >= GRAY_SAMPLE_HIGH)
                data |= (unsigned char)(0x80 >> i);
        }
        else
        {
            if (high_cnt < GRAY_SAMPLE_HIGH)
                data |= (unsigned char)(0x80 >> i);
        }
    }
    return data;
}

/* 中间6路(S2~S7)任意一路亮=有线，掩码0x7E */
char gray_line_detected(void)
{
    return (gray_read() & 0x7E) ? 1 : 0;
}

/* 8路任意一路检测到黑线=1 */
char gray_any_line_detected(void)
{
    return gray_read() ? 1 : 0;
}

/* 线偏向：-1=左，0=中，1=右 */
signed char gray_line_side(void)
{
    unsigned char data       = gray_read();
    unsigned char left_bits  = (data >> 4) & 0x07;   /* S2~S4 */
    unsigned char mid_bits   = (data >> 3) & 0x03;   /* S4~S5 */
    unsigned char right_bits = (data >> 1) & 0x07;   /* S5~S7 */
    unsigned char lc = 0, rc = 0, mask;

    if (!data)
        return g_gray_last_side;

    if (data & 0x80) {          /* S1最左边检测到线，强左修正 */
        g_gray_last_side = -2;
        return -2;
    }
    if (data & 0x01) {          /* S8最右边检测到线，强右修正 */
        g_gray_last_side = 2;
        return 2;
    }
```

```
    for (mask = 0x04; mask; mask >>= 1) {
        if (left_bits  & mask) lc++;
        if (right_bits & mask) rc++;
    }

    if (mid_bits == 0x03 || lc == rc) return 0;

    if (lc > rc) {
        g_gray_last_side = -1;
        return -1;
    }

    g_gray_last_side = 1;
    return 1;
}


signed char gray_line_side_from_data(unsigned char data)
{
    unsigned char left_bits  = (data >> 4) & 0x07;   /* S2~S4 */
    unsigned char mid_bits   = (data >> 3) & 0x03;   /* S4~S5 */
    unsigned char right_bits = (data >> 1) & 0x07;   /* S5~S7 */
    unsigned char lc = 0, rc = 0, mask;

    if (!data)
        return g_gray_last_side;

    if (data & 0x80) {
        g_gray_last_side = -2;
        return -2;
    }
    if (data & 0x01) {
        g_gray_last_side = 2;
        return 2;
    }

    for (mask = 0x04; mask; mask >>= 1) {
        if (left_bits  & mask) lc++;
        if (right_bits & mask) rc++;
    }
```

```
    if (mid_bits == 0x03 || lc == rc) return 0;

    if (lc > rc) {
        g_gray_last_side = -1;
        return -1;
    }

    g_gray_last_side = 1;
    return 1;
}

void arc_control_side(signed char side)
{
    if      (side ==  0) motor_forward(ARC_FAST, ARC_FAST);
    else if (side == -2) motor_forward(ARC_HARD_SLOW, ARC_HARD_FAST);
    else if (side ==  2) motor_forward(ARC_HARD_FAST, ARC_HARD_SLOW);
    else if (side == -1) motor_forward(ARC_SLOW, ARC_FAST);
    else                 motor_forward(ARC_FAST, ARC_SLOW);
}/* ============================================================
 * 声光控制
 * ============================================================ */
/* 到点提示：RD1亮（清零），蜂鸣器响；RD0保持状态灯 */
void beep_led_on(void)
{
    BUZZER = 1;
    PORTD  = (unsigned char)((PORTD | 0xFC) & ~0x02);
}

/* 提示结束：RD1灭（置1），蜂鸣器停；RD0保持状态灯 */
void beep_led_off(void)
{
    BUZZER = 0;
    PORTD  = (unsigned char)(PORTD | 0x02);
}

/* 两灯方案：运行中只亮RD0，RD1留给到点提示 */
void led_show_task(char task_num)
{
    task_num = task_num;
```

```
/* 待机闪烁：RD0每500ms切换一次 */
void idle_blink_update(void)
{
    if ((unsigned int)(g_timer_ms - g_idle_blink_ts) >= 500)
    {
        g_idle_blink_ts    = g_timer_ms;
        g_idle_blink_state = !g_idle_blink_state;
        if (g_idle_blink_state)
            PORTD = (unsigned char)(PORTD & ~0x01); /* RD0清零=亮 */
        else
            PORTD = (unsigned char)(PORTD |  0x01); /* RD0置1=灭 */
    }
}
```

3.2.4 循迹路径执行模块

循迹路径执行模块是小车运动控制的核心，主要由直线控制、弧线控制、原地旋转控制和路径段切换组成。程序不是把整条路线写成固定顺序的长代码，而是通过 g_route、g_route_len 和 g_step 记录当前任务路径表以及正在执行的段号，再由 runner_run()根据当前 Segment 类型分配给 run_spin()、run_straight()、run_arc()或 run_stop()处理。

直线段由 run_straight()执行。小车进入直线段后，以 JY62 陀螺仪的 Yaw 值作为方向基准 g_yaw_ref，当接收到新的姿态帧时调用 straight_control()进行 P 控制。straight_control_base()先计算当前 Yaw 与基准 Yaw 之间的误差，超过死区后按 KP 比例换算为 PWM 修正量，再让左右轮分别加减该修正量。若车头偏向一侧，左右轮速度差会把车头逐渐拉回目标方向，因此直线段不依赖黑线连续引导，也

能保持较稳定的行驶方向。

直线段的结束条件由灰度传感器判断。当小车刚进入直线段时，程序先用 STRAIGHT_IGNORE_MS 忽略一小段时间内的黑线，防止刚离开上一点时重复判定；随后如果 gray_line_detected()检测到黑线，说明到达下一个点或进入弧线入口，程序记录入弧 Yaw 基准、设置弧线相关状态，再调用 enter_beep_and_next()停车提示并切换到下一段。

弧线段由 run_arc()执行。程序先读取灰度传感器数据，得到当前黑线偏移方向，再调用 arc_control_side()调整左右轮速度。普通偏移时使用 ARC_FAST 和 ARC_SLOW 形成较温和的差速，边缘传感器触发时使用 ARC_HARD_FAST 和 ARC_HARD_SLOW 进行强修正，使小车能够沿弧线持续贴线行驶。

为了避免弧线刚开始时误判终点，run_arc()加入了入弧预转向、角度阈值和连续丢线确认三重保护。任务二入弧时先主动右转一小段角度，任务三 C 到 B 弧线先主动左转一小段角度，避免小车刚到点后直冲；随后程序要求弧线转角达到 ARC_MIN_YAW_RAW 之后，才允许根据全白状态判断出弧；最后还要求连续 ARC_LOST_CONFIRM 次读到全白，才认为已经离开黑线并到达目标点。这样可以减少单次抖动或局部断线造成的误判。

原地旋转段由 run_spin()执行，主要用于任务三和任务四起始时从 A 点对准 AC 方向。程序记录启动时的 Yaw 作为旋转基准 g_spin_yaw_ref，再比较当前 Yaw 偏转量与目标偏转角 AC_SPIN_TARGET 之间的差值；当误差大于容差时，根据剩余角度方向控制小车左旋或右旋；当误差进入 SPIN_TOLERANCE 范围后，更新直线行驶基准，并进入下一段。

每一段结束时都通过 enter_beep_and_next()进行统一处理：先停车，再点亮提示 LED 并启动蜂鸣器，同时把 g_step 推进到下一段。蜂鸣期间 runner_run()不会继续执行运动段，而是等待 beep_update()在设定时间后关闭蜂鸣和提示灯。这样到点提示、路径推进和下一段启动被统一管理，减少了各个路径段之间重复代码。

```
* ============================================================
* 直线P控制
* ============================================================ */
oid straight_control_base(unsigned int base_pwm, int max_corr)

    int error        = yaw_diff(g_yaw, g_yaw_ref);
    int error_scaled = 0;
    int correction   = 0;
    int left_pwm;
    int right_pwm;

    if (abs_val(error) > STRAIGHT_DEADBAND)
    {
        error_scaled = error / STRAIGHT_ERR_DIV;
        correction   = KP * error_scaled;

        if (correction >  max_corr) correction =  max_corr;
        if (correction < -max_corr) correction = -max_corr;
    }

    left_pwm  = (int)base_pwm + correction;
    right_pwm = (int)base_pwm - correction;

    if (left_pwm  > 1000) left_pwm  = 1000;
    if (left_pwm  < 100 ) left_pwm  = 100;

    if (right_pwm > 1000) right_pwm = 1000;
    if (right_pwm < 100 ) right_pwm = 100;

    motor_forward((unsigned int)left_pwm, (unsigned int)right_pwm);
}

void straight_control(void)
{
    straight_control_base(BASE_PWM, STRAIGHT_MAX_CORR);
}

/* ============================================================
 * 弧线循迹控制
 * ============================================================ */
void arc_control(void)
{
    arc_control_side(gray_line_side());
}

/* ============================================================
 * 进入下一段的统一处理
 * 停车，声光提示，step推进
 * ============================================================ */
void enter_beep_and_next(void)
{
    const Segment *seg = &g_route[g_step];

    motor_stop();
    beep_led_on();
    g_beep_start = g_timer_ms;
    g_in_beep    = 1;   /* 标志位：正在BEEP，runner_run等待 */
    g_step++;           /* 预先推进，BEEP结束后直接执行新段 */
    if (g_step < g_route_len && g_route[g_step].type == SEG_STRAIGHT)
    {
        g_straight_wait_clear = 1;
        g_straight_start_ms = g_timer_ms;
    }
}
```

```
/* ============================================================
 * 段执行器：run_spin
 * 原地旋转到目标角度（相对于g_spin_yaw_ref的偏转量）
 * ============================================================ */
void run_spin(void)
{
    const Segment *seg = &g_route[g_step];
    int current_diff   = yaw_diff(g_yaw, g_spin_yaw_ref);
    int remaining      = seg->spin_target - current_diff;

    if (abs_val(remaining) <= SPIN_TOLERANCE)
    {
        /* 到达目标角度，用绝对目标角作为直线基准，消除旋转停止误差 */
        GIE = 0;
        g_yaw_ref = yaw_add_raw(g_yaw_init, seg->spin_target);
        GIE = 1;
        enter_beep_and_next();
    }
    else if (remaining < 0)
    {
        /* 还需要顺时针转（Yaw减小方向） */
        motor_spin_right(SPIN_SPEED);
    }
    else
    {
        /* 还需要逆时针转（Yaw增大方向） */
        motor_spin_left(SPIN_SPEED);
    }
}

/* ============================================================
 * 段执行器：run_straight
 * 直线行驶，循迹检测到线时切换
 * ============================================================ */
void run_straight(void)
{
    if (g_new_frame) {
        g_new_frame = 0;
        if (g_align_cd)
        {
            straight_control_base(ALIGN_BASE_PWM, ALIGN_MAX_CORR);

            if (abs_val(yaw_diff(g_yaw, g_yaw_ref)) <= ALIGN_DONE_RAW)
                g_align_cd = 0;
        }
        else
        {
            straight_control();
        }
    }

    if ((unsigned int)(g_timer_ms - g_straight_start_ms) < STRAIGHT_IGNORE_MS)
        return;

    if (g_straight_wait_clear)
    {
        if (gray_line_detected())
            return;
        g_straight_wait_clear = 0;
    }

    /* 循迹检测到线：到达下一个点 */
    if (gray_line_detected())
    {
        GIE = 0;
        if (g_mode == MODE_TASK2 && g_step == 0)
            g_ab_yaw_ref = g_yaw_ref;
        g_arc_yaw_ref = g_yaw;  /* 记录入弧Yaw基准 */
        g_arc_lost_cnt = 0;
        g_arc_turn_ready = 0;
        if ((g_mode == MODE_TASK3 || g_mode == MODE_TASK4) && g_step == 1)
            g_gray_last_side = -1;   /* 任务三C->B默认向左找线 */
        else
            g_gray_last_side = 1;
        GIE = 1;
        enter_beep_and_next();
    }
}
```

```
/* ============================================================
 * 段执行器：run_arc
 * 弧线跟线，转角足够且连续全离线时切换
 * ============================================================ */
void run_arc(void)
{
    const Segment *seg = &g_route[g_step];
    unsigned int arc_turn = yaw_abs_diff(g_yaw, g_arc_yaw_ref);
    unsigned char gray = gray_read();
    char all_white = (gray == 0) ? 1 : 0;
    signed char side = gray_line_side_from_data(gray);

    if (g_mode == MODE_TASK2 && arc_turn < ARC_ENTRY_YAW_RAW)
    {
        motor_forward(ARC_FAST, ARC_SLOW);  /* B->C、D->A入弧先主动右转 */
        return;
    }

    if ((g_mode == MODE_TASK3 || g_mode == MODE_TASK4)
        && seg->label == 'B'
        && arc_turn < ARC_ENTRY_CB_YAW_RAW)
    {
        motor_forward(ARC_SLOW, ARC_FAST);  /* C->B入弧先主动左转 */
        return;
    }

    if (!g_arc_turn_ready)
    {
        if (arc_turn < ARC_MIN_YAW_RAW)
        {
            arc_control_side(side);
            return;
        }
        g_arc_turn_ready = 1;
    }

    if (!all_white)
    {
        g_arc_lost_cnt = 0;
        arc_control_side(side);
        return;
```

```
        arc_control_side(side);
        return;
    }

    if (g_arc_lost_cnt < ARC_LOST_CONFIRM)
    {
        g_arc_lost_cnt++;
        arc_control_side(side);
        return;
    }

    /* 转角达到过阈值，并且8路灰度连续全白后，认为已经出弧到点 */
    GIE = 0;
    if (g_mode == MODE_TASK2 && seg->label == 'C')
    {
        g_yaw_ref = yaw_opposite(g_ab_yaw_ref);  /* C->D：AB实际方向+180度 */
        g_align_cd = 1;
    }
    else if ((g_mode == MODE_TASK3 || g_mode == MODE_TASK4) && seg->label == 'B')
    {
        g_yaw_ref = g_bd_yaw_ref;                /* B->D：任务三理论斜线方向 */
        g_align_cd = 1;
    }
    else
    {
        g_yaw_ref = g_yaw;
        g_align_cd = 0;
    }
    GIE = 1;
    enter_beep_and_next();


/* ============================================================
 * 段执行器：run_stop
 * ============================================================ */
void run_stop(void)
```

```
    /* 任务四：判断圈数 */
    if (g_mode == MODE_TASK4)
    {
        g_lap++;
        if (g_lap < 4)
        {
            /* 未满4圈，重置路径从第一段重新跑 */
            g_step = 0;
            GIE = 0;
            g_spin_yaw_ref = g_yaw_init;
            g_yaw_ref      = g_yaw_init;
            g_arc_yaw_ref  = g_yaw_init;
            g_bd_yaw_ref   = yaw_add_raw(g_yaw_init, -26000);
            g_arc_lost_cnt = 0;
            g_arc_turn_ready = 0;
            g_gray_last_side = 1;
            g_align_cd = 0;
            g_straight_wait_clear = 0;
GIE = 1;
            return;
        }
    }
    /* 完成 */
    motor_stop();
    BUZZER = 1;
    PORTD  = 0xFC;      /* 完成：RD0/RD1亮，其余灭 */
    g_mode = MODE_DONE;
}

/* ============================================================
 * BEEP状态处理
 * ============================================================ */
void beep_update(void)
{
    if (!g_in_beep) return;
    if ((unsigned int)(g_timer_ms - g_beep_start) >= BEEP_TIME_MS)
    {
        beep_led_off();
        g_in_beep = 0;
        /* BEEP结束，g step已在enter beep and next里推进，继续执行新段
```

3.2.5 电机驱动模块

电机驱动模块负责把上层控制算法输出的“速度和方向”转换为 TB6612 电机驱动芯片能够识别的 PWM 和方向电平。小车左右轮分别由两路 PWM 控制速度，由 RB4 到 RB7 控制方向，程序通过 pwm_set()、motor_forward()、motor_spin_left()、motor_spin_right()和 motor_stop()等函数完成具体驱动。

pwm_set()是底层速度输出函数，输入左右轮的 PWM 值后，先把数值限制在 0 到 1000 范围内，再写入 CCP1 和 CCP2 相关寄存器。由于 PWM 寄存器由高 8 位和低 2 位共同组成，程序在函数内部完成拆分写入，上层模块只需要传入直观的速度数值即可。

motor_forward()用于控制小车前进，它设置左右电机方向引脚为正转方向，再调用 pwm_set()输出左右轮速度。在直线控制中，左右轮速度接近；在弧线或姿态修正中，左右轮速度会产生差值，小车就会向速度较慢的一侧转向，实现差速循迹。

motor_spin_left()和 motor_spin_right()用于原地旋转。它们让左右电机方向相反，并使用相同旋转速度 SPIN_SPEED，使小车围绕自身中心转动，适合任务三开始时快速对准目标方向。motor_stop()则把 PWM 占空比清零并关闭方向输出，用于到点提示、任务完成和复位停车。

该模块的特点是接口简单、层次清楚。路径执行模块只关心“前进、左旋、右旋、停止”和左右轮速度，不直接操作寄存器；寄存器配置和方向电平全部封装在电机驱动模块内部。这样既方便调试速度参数，也方便在电机接线或驱动芯片逻辑变化时集中修改。

```
void pwm_init(void)
{
    TRISC1  = 0;
    TRISC2  = 0;
    PR2     = 249;
    T2CON   = 0x04;
    CCP1CON = 0x0C;
    CCPR1L  = 0;
    CCP2CON = 0x0C;
    CCPR2L  = 0;
}

void pwm_set(unsigned int left, unsigned int right)
{
    if (left  > 1000) left  = 1000;
    if (right > 1000) right = 1000;
    CCPR1L  = (unsigned char)(left  >> 2);
    CCP1CON = (CCP1CON & 0xCF) | (unsigned char)((left  & 0x03) << 4);
    CCPR2L  = (unsigned char)(right >> 2);
    CCP2CON = (CCP2CON & 0xCF) | (unsigned char)((right & 0x03) << 4);
}
```

```
 * ============================================================
/* 两轮前进 */
void motor_forward(unsigned int l, unsigned int r)
{
    AIN1 = 1; AIN2 = 0;
    BIN1 = 1; BIN2 = 0;
    pwm_set(l, r);
}

/* 原地左转：左轮后退，右轮前进 */
void motor_spin_left(unsigned int spd)
{
    AIN1 = 0; AIN2 = 1;    /* 左轮反转 */
    BIN1 = 1; BIN2 = 0;    /* 右轮正转 */
    pwm_set(spd, spd);
}

/* 原地右转：左轮前进，右轮后退 */
void motor_spin_right(unsigned int spd)
{
    AIN1 = 1; AIN2 = 0;    /* 左轮正转 */
    BIN1 = 0; BIN2 = 1;    /* 右轮反转 */
    pwm_set(spd, spd);
}

void motor_stop(void)
{
    AIN1 = 0; AIN2 = 0;
    BIN1 = 0; BIN2 = 0;
    pwm_set(0, 0);
}

void motor_init(void)
{
    TRISB4 = 0; TRISB5 = 0;
    TRISB6 = 0; TRISB7 = 0;
    motor_stop();
}
```

3.2.6 主程序与任务调度模块

主程序与任务调度模块负责把各个功能模块组织成完整运行流程，是整个软件的控制中心。main()函数先调用 init_all()完成系统初始化，然后进入 while(1)循环，不断执行按键扫描、待机显示和路径运行。程序整体采用状态机思想，通过 g_mode 区分待机、任务运行和完成状态，通过 g_step 区分当前执行到路径表中的哪一段。

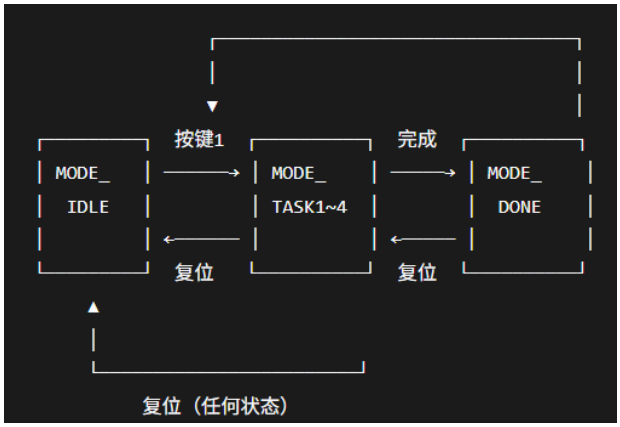
key_scan()负责按键交互。RB0 为复位键，在任何状态下都有效，按下后立即停车、关闭蜂鸣器、熄灭 LED 并回到待机状态；RB1 启动任务一，RB2 启动任务二，RB3 短按启动任务三，长按超过 LONG_PRESS_MS 启动任务四。程序使用 g_timer_ms 进行 20ms 去抖，并用按下时间判断短按和长按，避免机械按键抖动导致重复触发。

start_task()是任务启动的公共入口。它首先确认当前处于 MODE_IDLE，防止运行中误触发新任务；随后清零路径段号、圈数、蜂鸣状态、弧线丢线计数和各类控制标志；再等待 JY62 连续接收若干帧稳定数据，把当前 Yaw 记录为初始基准。最后根据按键选择不同路径表，把 g_route 指向 route1、route2 或 route3，并把 g_mode 切换为对应任务模式。

runner_run()是路径执行器的主入口。如果当前正在蜂鸣提示，则优先调用 beep_update()等待提示结束；如果没有提示且路径段号有效，则根据 g_route[g_step].type 选择执行原地旋转、直线、弧线或停止函数。这样主循环不需要知道每个任务的具体路线细节，只需要反复调用 runner_run()，具体动作由路径表和段执行函数共同决定。

任务四在 run_stop()中做了特殊处理：由于任务四要求任务三路线重复四圈，程序每次执行到 SEG_STOP 时先增加圈数 g_lap，若未满四圈，就把 g_step 重新置零并重新设置 Yaw 基准、弧线状态和对角线方向基准，继续执行下一圈；当圈数达到要求后，才真正停车并进入 MODE_DONE 状态。

声光提示也由调度模块统一管理。待机时 RD0 慢闪表示系统就绪；任务运行中 LED 显示运行状态；到达各关键点时 enter_beep_and_next()触发蜂鸣和提示灯；任务完成后电机停止并保持完成提示，等待复位键。通过状态机、路径表、按键扫描和声光提示的配合，程序实现了从用户选择任务到小车自动行驶、到点提示、任务完成复位的完整闭环。



```c
/* ============================================================
 * 路径执行器主入口
 * ============================================================ */
void runner_run(void)
{
    /* 正在BEEP中，等待结束 */
    if (g_in_beep)
    {
        beep_update();
        return;
    }

    /* 防止step越界 */
    if (g_step >= g_route_len) return;

    switch (g_route[g_step].type)
    {
    case SEG_SPIN:     run_spin();     break;
    case SEG_STRAIGHT: run_straight(); break;
    case SEG_ARC:      run_arc();      break;
    case SEG_STOP:     run_stop();     break;
    }
}
```

```c
/* ============================================================
 * 启动任务公共处理
 * ============================================================ */
void start_task(char task_num)
{
    unsigned char stable_cnt;

    /* 只在待机状态下响应 */
    if (g_mode != MODE_IDLE) return;

    g_step     = 0;
    g_lap      = 0;
    g_in_beep  = 0;
    g_arc_lost_cnt = 0;
    g_arc_turn_ready = 0;
    g_gray_last_side = 1;
    g_align_cd = 0;
    g_straight_wait_clear = 0;
    g_straight_start_ms = g_timer_ms;

    /* 等待JY62连续5帧稳定 */
    g_new_frame = 0;
    stable_cnt  = 0;
    while (stable_cnt < 5) {
        if (g_new_frame) { g_new_frame = 0; stable_cnt++; }
    }

    /* 记录绝对初始基准 */
    GIE = 0;
    g_yaw_init     = g_yaw;
    g_yaw_ref      = g_yaw;
    g_spin_yaw_ref = g_yaw;
    g_arc_yaw_ref  = g_yaw;
    g_ab_yaw_ref   = g_yaw;
    g_bd_yaw_ref   = yaw_add_raw(g_yaw_init, -26000);
    GIE = 1;

    /* 清LED，显示任务编号 */
    PORTD = 0xFF;                /* 先全灭 */
```

```
switch (task_num)
{
case 1:
    g_route     = route1;
    g_route_len = ROUTE1_LEN;
    g_mode      = MODE_TASK1;
    break;
case 2:
    g_route     = route2;
    g_route_len = ROUTE2_LEN;
    g_mode      = MODE_TASK2;
    break;
case 3:
    g_route     = route3;
    g_route_len = ROUTE3_LEN;
    g_mode      = MODE_TASK3;
    break;
case 4:
    g_route     = route3;
    g_route_len = ROUTE3_LEN;
    g_mode      = MODE_TASK4;
    break;
}
```

```
/* ============================================================
 * 按键扫描
 * RB0：复位（任何状态）
 * RB1：启动任务一
 * RB2：启动任务二
 * RB3：短按=任务三，长按>1s=任务四
 * 低电平有效，时间戳去抖20ms
 * ============================================================ */
#define DEBOUNCE_MS  20

static char          btn_reset_down  = 0;
static char          btn_t1_down     = 0;
static char          btn_t2_down     = 0;
static char          btn_t3_down     = 0;
static unsigned int  btn_reset_ts    = 0;
static unsigned int  btn_t1_ts       = 0;
static unsigned int  btn_t2_ts       = 0;
static unsigned int  btn_t3_ts       = 0;
static unsigned int  btn_t3_hold_ts  = 0;
static char          btn_t3_fired    = 0;    /* 长按已触发，避免重复 */
static char          btn_t3_confirmed= 0;    /* 短按已确认，避免重复 */

void key_scan(void)
{
    /* ---- RB0 复位（任何状态有效） ---- */
    if (!BTN_RESET) {
        if (!btn_reset_down) {
            btn_reset_down = 1;
            btn_reset_ts   = g_timer_ms;
        }
        else if ((unsigned int)(g_timer_ms - btn_reset_ts) >= DEBOUNCE_MS)
            /* 确认按下：停车，回待机，LED全灭等待闪烁 */
            motor_stop();
            BUZZER        = 0;
            PORTD         = 0xFF;
            g_in_beep     = 0;
            g_arc_lost_cnt = 0;
            g_arc_turn_ready = 0;
            g_gray_last_side = 1;
```

```
            g_idle_blink_state = 0;
            btn_reset_ts  = g_timer_ms + 30000; /* 防止持续触发 */
        }
    } else { btn_reset_down = 0; }

    /* 任务运行中或完成后只响应复位键 */
    if (g_mode != MODE_IDLE) return;

    /* ---- RB1 启动任务一 ---- */
    if (!BTN_TASK1) {
        if (!btn_t1_down) {
            btn_t1_down = 1;
            btn_t1_ts   = g_timer_ms;
        }
        else if ((unsigned int)(g_timer_ms - btn_t1_ts) >= DEBOUNCE_MS) {
            start_task(1);
            btn_t1_ts = g_timer_ms + 30000;
        }
    } else { btn_t1_down = 0; }

    /* ---- RB2 启动任务二 ---- */
    if (!BTN_TASK2) {
        if (!btn_t2_down) {
            btn_t2_down = 1;
            btn_t2_ts   = g_timer_ms;
        }
        else if ((unsigned int)(g_timer_ms - btn_t2_ts) >= DEBOUNCE_MS) {
            start_task(2);
            btn_t2_ts = g_timer_ms + 30000;
        }
    } else { btn_t2_down = 0; }

    /* ---- RB3 启动任务三（长按=任务四） ---- */
    if (!BTN_TASK3) {
        if (!btn_t3_down) {
            btn_t3_down      = 1;
            btn_t3_ts        = g_timer_ms;
            btn_t3_hold_ts   = g_timer_ms;
            btn_t3_fired     = 0;
            btn_t3_confirmed = 0;
        }
        /* 长按优先：超过1s升级为任务四 */
        if (!btn_t3_fired
            && (unsigned int)(g_timer_ms - btn_t3_hold_ts) >= LONG_PRESS_MS) {
            btn_t3_fired     = 1;
            btn_t3_confirmed = 1;
            start_task(4);
        }
    }
    else {
        /* 松开时：若未触发长按且已去抖确认，启动任务三 */
        if (btn_t3_down && !btn_t3_fired && !btn_t3_confirmed
            && (unsigned int)(g_timer_ms - btn_t3_ts) >= DEBOUNCE_MS) {
            start_task(3);
        }
        btn_t3_down      = 0;
        btn_t3_fired     = 0;
        btn_t3_confirmed = 0;
    }
}


/* ============================================================
 * 系统初始化
 * ============================================================ */
void init_all(void)
{
    CMCON  = 0x07;      /* 关比较器 */
    ADCON1 = 0x06;      /* RA全部数字IO */

    /* RA: RA0输入(灰度OUT), RA1~RA3输出(灰度AD0~AD2), 其余输出 */
    TRISA  = 0x01;
    /* RB: RB0~RB3输入(按键), RB4~RB7输出(电机方向) */
    TRISB  = 0x0F;
    TRISD  = 0x00;      /* RD全部输出(LED) */
    TRISE  = 0x00;      /* RE输出(蜂鸣器) */

    /* 使能RB口内部上拉（OPTION_REG的RBPU位清零） */
    RBPU = 0;

    PORTD  = 0xFF;      /* 高电平=全灭（低电平点亮） */
```

```
    GRAY_AD0 = 0;
    GRAY_AD1 = 0;
    GRAY_AD2 = 0;

    pwm_init();
    motor_init();
    uart_init();
    timer1_init();

    /* 待机状态，RD0开始闪烁 */
    g_idle_blink_ts    = 0;
    g_idle_blink_state = 0;


* ================================================================
* 主程序
* ================================================================ */
oid main(void)

    init_all();

    while (1)
    {
        key_scan();

        if (g_mode == MODE_IDLE)
            idle_blink_update();
        else if (g_mode != MODE_DONE)
            runner_run();
        /* MODE_DONE：全亮等复位，不需要额外处理 */
    }
```

3.2.7 JY62 陀螺仪数据处理模块

JY62 陀螺仪模块主要为小车提供 Yaw 偏航角数据，用于直线段姿态修正、原地旋转角度判断以及弧线段转角确认。程序没有直接使用延时控制转向，而是通过串口实时接收陀螺仪姿态帧，把当前车头方向转换成可比较的角度基准，从而提高直线行驶和转向到位的稳定性。

（1）串口初始化：uart_init()将 RC7 设置为 USART 接收端，RC6 设置为发送端，配置异步串口通信并开启接收中断。JY62 的 TX 引脚接到 PIC16F877A 的 RC7 后，陀螺仪会持续发送姿态数据，单片机通过中断方式接收数据，避免主循环长时间等待串口数据而影响按键扫描和路径执行。

（2）数据帧接收与校验：在 interrupt isr()中，程序先判断 RCIF 和 RCIE，确认 USART 接收中断有效，然后从 RCREG 读取一个字节。接收时先寻找帧头 0x55，随后要求第二个字节为 0x53，表示当前帧为角度数据帧；当接收满 FRAME_LEN 个字节后，对前 10 个字节求和并与最后一个校验字节比较。只有校验正确时，程序才把第 6、7 字节组合为 Yaw 原始值并写入 g_yaw，同时置位 g_new_frame，表示有新的姿态数据可供控制算法使用。

（3）Yaw 角度差处理：由于 JY62 输出的 Yaw 值存在正负范围和边界回绕问题，程序设计了 yaw_diff()函数，把当前 Yaw 与参考 Yaw 的差值归一化到-32768 到

+32767 范围内，对应实际角度约-180 度到+180 度。这样即使车头角度跨过正负边界，程序仍能得到正确的最短角度差。yaw_add_raw()用于在初始角度基础上叠加目标偏转量，yaw_opposite()用于得到相反方向的角度基准，yaw_abs_diff()用于计算弧线段已经转过的绝对角度。

（4）直线姿态修正：在 run_straight()中，程序只有在 g_new_frame 置位时才进行一次直线控制，说明控制动作跟随陀螺仪新数据更新。straight_control_base()通过 yaw_diff(g_yaw, g_yaw_ref)计算当前方向误差，误差超过死区后按 KP 比例换算成 PWM 修正量，再让左、右轮速度分别加减该修正量。若车头向一侧偏转，左右轮差速会自动把车头拉回基准方向，从而使直线段更加稳定。

（5）原地旋转与任务基准记录：任务启动时，start_task()会先等待 JY62 连续接收 5 帧稳定数据，再把当前 g_yaw 记录为 g_yaw_init、g_yaw_ref、g_spin_yaw_ref 等基准值。任务三和任务四开始时，run_spin()用 yaw_diff(g_yaw, g_spin_yaw_ref)计算当前已经旋转的角度，再与 AC_SPIN_TARGET 比较；当剩余角度进入 SPIN_TOLERANCE 范围内时，认为小车已经对准目标方向，并更新后续直线段的 Yaw 基准。

通过 JY62 模块，程序把小车的运动控制从单纯依靠时间和速度估计，改为根据实际车头角度闭环修正。直线段利用 Yaw 误差进行 P 控制，旋转段利用目标角判断到位，弧线段利用转角阈值防止误判终点，使整车在不同任务路径下的方向控制更加可靠。

```
/* 陀螺仪（USART中断更新） */
volatile int  g_yaw       = 0;
volatile char g_new_frame = 0;

/* USART接收缓冲 */
#define FRAME_LEN 11
volatile unsigned char g_uart_buf[FRAME_LEN];
volatile unsigned char g_uart_idx  = 0;
volatile char          g_uart_sync = 0;
```

```
/* Yaw角差，归一化到 -32768~+32767 (对应-180°~+180°) */
int yaw_diff(int current, int ref)
{
    long diff = (long)current - (long)ref;
    if (diff >  32767L) diff -= 65536L;
    if (diff < -32768L) diff += 65536L;
    return (int)diff;
}

int yaw_add_raw(int ref, int delta)
{
    int value = ref + delta;
    if (delta > 0 && value < ref) value -= 32768;
    if (delta < 0 && value > ref) value += 32768;
    return value;
}

/* 返回与ref相差180度的Yaw原始值 */

int yaw_opposite(int ref)
{
    if (ref >= 0) return ref - 32767 - 1;
    return ref + 32767 + 1;
}

unsigned int yaw_abs_diff(int current, int ref)
{
    long diff = (long)current - (long)ref;
    if (diff >  32767L) diff -= 65536L;
    if (diff < -32768L) diff += 65536L;
    if (diff < 0) diff = -diff;
    return (unsigned int)diff;
}
```

```
/* ============================================================
 * 段执行器：run_spin
 * 原地旋转到目标角度（相对于g_spin_yaw_ref的偏转量）
 * ============================================================ */
void run_spin(void)
{
    const Segment *seg = &g_route[g_step];
    int current_diff   = yaw_diff(g_yaw, g_spin_yaw_ref);
    int remaining      = seg->spin_target - current_diff;

    if (abs_val(remaining) <= SPIN_TOLERANCE)
    {
        /* 到达目标角度，用绝对目标角作为直线基准，消除旋转停止误差 */
        GIE = 0;
        g_yaw_ref = yaw_add_raw(g_yaw_init, seg->spin_target);
        GIE = 1;
        enter_beep_and_next();
    }
    else if (remaining < 0)
    {
        /* 还需要顺时针转（Yaw减小方向） */
        motor_spin_right(SPIN_SPEED);
    }
    else
    {
        /* 还需要逆时针转（Yaw增大方向） */
        motor_spin_left(SPIN_SPEED);
    }
}
```

四、整体设计完成情况说明

在本设计中，我们完成了系统初始化、灰度循迹、陀螺仪辅助直线控制、原地旋转、弧线循迹、多任务路径切换、按键交互、声光提示等功能，完成了四个任务的设计和执行，把各个模块都相对好的完成了，在任务执行中，按下对应按

键后，小车可根据选择执行不同路径：任务一为 A 点到 B 点直线行驶在 3 秒左右完成；任务二为矩形环路带弧线过渡行驶在 12 秒左右完成；任务三为对角线斜穿加弧线回环行驶在 13 秒左右完成；任务四为任务三路径重复四圈，实现多次环绕在 56 秒左右完成。

五、反思及总结

5.1小车设计的优势之处

这个设计最大的优势就是结构清晰、好调车、好扩展。首先，代码把硬件初始化、传感器采集、运动控制和任务调度全部分开，可以分开修改部分模块出现的问题，所有像速度、转弯灵敏度、P 控制系数这些需要反复调试的参数，都统一放在开头用宏定义，现场调车只需要改数字，不用翻代码逻辑。再者，任务路径是用路径表来描述的，想加新任务只需要写个数组，底层代码完全不用动，扩展很方便。控制算法上也考虑得很周全，直线用陀螺仪做 P 控制，跑得直，抗干扰性比较强；循迹分了普通修正和边缘强修正，不容易丢线；到点判定要同时满足转够了角度和连续全白两个条件，不会中途误判。

遇到的问题

1. 弧线端点识别不稳定

问题：弧线末端 B/C/A 点最开始只靠灰度“全白”判断。实际运行中，全白可能表示真正到达端点，也可能表示弧线中途脱轨，容易导致提前切段或漏判。
解决思路：将端点识别改为组合条件：
转角达到阈值 + 灰度连续全白 = 判定端点
角度用于判断是否进入允许判定状态，只有角度满足条件后，灰度全白才作为有效判定依据。
中间尝试过只用角度判断，但会出现“灰度还没全白就提前判点”；也尝试过全白连续确认，但如果角度阈值没到又会漏判。最后思路是把角度作为“允许判定状态”，只有角度到位后，灰度全白才有效。

2. 弧线中途脱轨与真正端点难以区分

问题：脱轨和端点都可能表现为 8 路全白。如果直接将全白判定为到点，会导致路径状态错乱。
解决思路：明确区分“全白”和“到点”：
全白 ≠ 到点
全白仅表示疑似出线状态
优化逻辑：先判断角度是否接近端点，再判断是否全白；必要时加入找线逻辑，通过能否重新找到黑线来区分脱轨与真正到点。如果继续优化可能的想法： 全白后先低速向预期方向找线，如果能找回黑线，说明只是脱轨；如果找不回且角度也满足，才判端点。

3. 直线陀螺仪修正过敏导致左右摆动

问题：直线段利用 JY62 保持方向时，修正过于灵敏，导致左右轮差速过大，车辆反复过冲摆动。

解决方法：对 P 控制增加三层限制：

1. 设置死区，小角度误差不修正；
2. 对误差进行缩放，避免误差直接放大成 PWM 差值
3. 限制最大修正量，防止单次修正过大

核心目标是允许小误差存在，并逐步修正，减少过冲。

4. 弧线切换到直线时姿态不稳定

问题：从 C 点或 B 点出弯后，小车车头方向经常不是理想直线方向。如果直接进入正常直线速度，陀螺仪会用较大差速强行拉正，表现为突然大弯，甚至偏出轨道

解决方法：增加低速方向校正阶段：进入直线段后先低速修正方向，当角度误差小于阈值后，再恢复正常速度运行直线控制。

代码中对应的是类似 g_align_cd 的状态。它的作用是让小车在 C→D、B→D 这种出弯后的直线段，先低速把车头拉正，避免刚出弯就高速大幅修正。

5. 任务四多圈运行时角度基准累计误差

问题：多圈运行时，若每圈都以当前姿态重新建立角度基准，会将上一圈误差持续累积到下一圈。

解决思路：

首先尝试使用 一开始每圈回到 A 点后，用当前 g_yaw 作为下一圈的新基准；也就是每圈重置当前基准，能够抵抗漂移，但会继承姿态误差。

最后还是用最简单的 全程固定第一次启动时的初始基准（g_yaw_init），避免误差逐圈传递。

这样每圈的 A→C、B→D 都基于同一个初始零度，不会把上一圈回 A 的姿态误差继续传递。并通过实验验证大幅转向不是由基准累积误差导致，所以不需要根据上一圈的结束点方向作为初始状态，直接用最开始开机时刻的零点一直作为参照方向就可以

5.2缺陷及需要改进之处

1) 弧线入弧角度固定

用固定角度预转向，适应性差，如果开始时放置的角度稍有偏差，就有可能导致后续在转弯过后无法识别到轨迹，从而路线发生偏移

2) 无传感器故障检测

传感器异常时小车仍继续行驶，可以增加一个程序当行驶异常时，小车停止行进。

3) 陀螺仪漂移未处理

长时间运行角度基准可能偏移，需要定期检测静止状态并重新校准零点

六、成员分工、贡献及心得体会

6.1 分工贡献

1) 组员：林祖澎
主要负责整体思路构建及软硬件调试等。
2) 组员：吴智聪
主要负责硬件辅助调试等。
3) 组员：谢欣怡
主要负责记录实验数据，实验报告编写等。
4) 组员：李哲
主要负责 ppt 制作等。

6.2 心得体会

1) 林祖澎
这次课程设计，它并不是简单地让小车动起来，而是要让程序能够根据传感器输入判断当前情况，并控制小车做出正确动作。刚开始面对整个任务时，会觉得流程比较多，但在设计过程中关键是先把复杂过程拆成清晰的一个个状态，让程序在每个时刻只处理当前最需要完成的事情。小车的运行过程需要不断判断 是否到达目标端点、是否需要转向等，这些判断如果没有明确条件，就很容易出现提前切换或无法切换的问题。同时，传感器的读数并不总是稳定的，要通过连续判断、延时确认等方式提高可靠性。这让我认识到，程序既要考虑代码逻辑，也要考虑硬件和环境带来的不确定性。
调试过程中，遇到问题不能盲目修改，而应该先判断问题大概出在输入、输出还是控制逻辑上。比如先确认传感器和按键等硬件，再分析程序状态或者是选择判断逻辑是否切换正确。这样一步步缩小范围，比直接猜测问题原因更有效。
这次课程设计让我学到的不只是单片机的使用，更重要的是学会了如何把一个实际控制问题转化成清晰、可靠的程序逻辑。它让我感受到嵌入式开发需要完整清晰的逻辑链条，也需要不断的试错测试，结合实际现象去验证自己的设计想法
2) 吴智聪
这次课设让我深刻认识到，在做小车自动驾驶时单纯依赖灰度传感器存在很大局限。在无引导线直道入弯时，车身偏差极易导致传感器误判转向。通过引入陀螺仪航向锁定和分段强制捕获策略，问题迎刃而解。还有交错行驶，仅靠单一的灰度传感器肯定是没办法实现的，哪怕加入了陀螺仪，理论上可行但是实际完全不一样，甚至陀螺仪不能帮助修正方向还会导致更多错误。这让我体会到多传感器融合与状态机思维在复杂控制中的关键作用，动手调试能力也提升明显。从长远来看，这个路线可以固定背图，但是到了现实当中存在许多不可控因素，应当通过更深入的学习专业理论知识与积累实际实践经验来完善自己。

3） 谢欣怡

在这次课程设计中，我们仍然还是延续上次数电课程设计的小组分配，有了第一次的配合经历，在本次课程设计的过程中，我们配合得更加默契，效率也有所提升，这次的课程设计主要是要完成小车自动驾驶的循迹以及让小车按照指定路线行驶，因为跟上次课程设计有不少相似之处，所以我们小组在硬件的搭建以及灰度传感器的调试过程中更加熟练了不少，在课程设计的过程中，需要调用不少我们在微机原理课程中所新学到的知识，而在这一方面，也暴露出了我对于课本内容的不够熟悉以及对于知识应用的不够全面，还是有许多内容需要更加完善的掌握。不过在这几次课程设计的过程中，也让我感受到了将所学知识应用在实际中的精彩体验。

4） 李哲

这次实验我负责电机 PWM 控制、直线行驶方向和原地旋转控制模块，一开始以为只是配置定时器、写简单控制逻辑就能搞定，实际调试才发现全是细节：PWM 频率不对会让电机抖动、左右电机的转速差不做标定就会跑偏，直线行驶的方向修正太猛会“蛇形跑”、太弱又会冲线，原地旋转没加减速逻辑会惯性过冲，最后靠反复校准 PWM 占空比、优化修正系数和加减速控制，才让车跑稳了。这次实验让我明白，底层驱动和控制逻辑从来不是写出来就行，必须结合硬件特性一遍遍调试优化，才能真正落地。